字符串

qwaszx

2026 年 7 月 29 日

目录

第零章 字符串基础与符号约定 **3**

0.1 基本符号 3

0.2 基本概念 4

第一章 字符串哈希 **5**

1.1 哈希函数及其碰撞概率 5

1.2 子串哈希 6

1.3 应用 6

第二章 Border，周期与字符串匹配 **7**

2.1 Border 与周期 7

2.2 字符串匹配 10

2.3 Z 函数 12

第三章 有限状态自动机 **14**

3.1 基本概念 14

3.2 等价类 16

3.3 在自动机上 DP 19

3.4 DAG 链剖分 20

第四章 Trie 和 AC 自动机 **21**

4.1 Trie 21

4.1.1 压缩 Trie 21

4.1.2 01 Trie 21

4.2 AC 自动机 22

4.2.1 字符串匹配 23

第五章 后缀结构 **26**

5.1 后缀树 26

5.1.1 构建 26

5.1.2 应用 28

5.1.3 广义后缀树 28

5.2 后缀数组 29

5.3 后缀自动机 29

5.3.1 构建 29

5.3.2 广义后缀自动机 30

5.3.3 应用 31

5.4 练习 32

5.5 基本子串结构 33
5.5.1 压缩后缀自动机 33

第六章 回文结构 34
6.1 Manacher 算法 34
6.2 回文树 35
6.3 回文与 Border 36

第七章 Lyndon Words 和 Runs 39

参考文献 40
7.1 Border，周期与字符串匹配 40
7.2 自动机 40
7.3 Trie 和 AC 自动机 40
7.4 后缀结构 40
7.5 回文结构 40

0 字符串基础与符号约定

本节约定使用的基本符号和定义。除非特别说明，字符串下标从 1 开始。

0.1 基本符号

Σ	字符集，通常假设为小写字母集合。一般来说可以是任意有限全序集。
$c \in \Sigma$	字符。
s, t, S, T	字符串，即有限个字符组成的序列。
ϵ	空串，长度为 0。
Σ^*	由 Σ 中的字符组成的所有字符串的集合。
$\|s\|$	字符串 s 的长度。
$s[i]$	s 的第 i 个字符。特别地，当 $i < 0$ 时表示 $s[\|s\| + i + 1]$，例如 $s[-1] = s[\|s\|]$。
$s[l, r]$、$s[l, r)$、$s(l, r)$ 等	子串，即区间内的 $s[i]$ 从左到右拼接形成的字符串。其中 $[l, r]$ 为闭区间，$[l, r)$ 不包含 r，(l, r) 不包含两端。端点为负数时按上面的规则换算，例如 $s[-r, -l] = s[\|s\| - r + 1, \|s\| - l + 1]$。$l, r$ 中间的逗号有时会使用冒号替代。
st 或 $s + t$	字符串拼接。
s^R	字符串 s 的反串，即 $s^R[i] = s[\|s\| - i + 1]$。
$s[: i]$	前缀 $s[1, i]$。
$s[i :]$	后缀 $s[i, \|s\|]$。

0.2　基本概念

前缀	若存在字符串 u 使 $t = su$，则称 s 是 t 的前缀。
后缀	若存在字符串 u 使 $t = us$，则称 s 是 t 的后缀。
子串	若存在字符串 u, v 使 $t = usv$，则称 s 是 t 的子串。
子序列	从 t 中删除若干字符后得到的字符串。子序列不要求连续。
$\mathrm{LCP}(s,t)$	s, t 的最长公共前缀长度。
$\mathrm{LCS}(s,t)$	s, t 的最长公共后缀长度。
字典序	从左到右比较第一处不同字符；若一者是另一者前缀，则较短者更小。
回文串	满足 $s = s^R$ 的字符串。

1 字符串哈希

1.1 哈希函数及其碰撞概率

字符串哈希是比较字符串最简单的方法。通常来说，我们使用如下的方式将字符串映射到 $\mathbb{F}_p$ 中的一个元素：

算法 1.1. 字符串哈希. 假设字符集 $\Sigma = \{1, \ldots, c\}$。对于素数 $p > |\Sigma|$ 和整数参数 $0 \le x < p$，字符串哈希函数定义为

$$f_x(S) = \left(\sum_{i=1}^{|S|} S[i] \cdot x^{|S|-i} \right) \bmod p$$

.

在使用时，随机选择 $0 \le x < p$，然后将全局的哈希函数固定为 $f = f_x$。

评注. 字符集不包含 0 是重要的，否则无法区分不同长度的字符串。

在 *OI* 中，x 常常会用一个**出题人猜不到的**固定常数替代随机数。

哈希函数也可以反过来定义为

$$f_x(S) = \left(\sum_{i=0}^{|S|-1} S[i+1] \cdot x^{i} \right) \bmod p.$$

如果 S, T 是相同的字符串，那么它们映射到同一个值；如果是不同的字符串，则它们大概率映射到不同的值。下面我们分析这个哈希函数的碰撞概率。

定理 1.2. 对任意两个不同的字符串 $S \neq T$，有 $\Pr[f(S) = f(T)] \le \frac{\max(|S|,|T|)-1}{p}$。

证明. 记 $l = \max(|S|, |T|)$。注意到 $f(S)$ 和 $f(T)$ 都是不超过 $l-1$ 次的关于 x 的多项式，$f(S) = f(T)$ 即是说这两个多项式的差为 0。由于 $S \neq T$，$f(S) - f(T)$ 是一个非零多项式，而 $\mathbb{F}_p$ 中的 d 次非零多项式至多有 d 个根，故 x 落在这个多项式的根上的概率不超过 $\frac{l-1}{p}$，即 $\Pr[f(S) = f(T)] \le \frac{\max(|S|,|T|)-1}{p}$。 □

推论 1.3. 对于 n 个长度不超过 l 的字符串，哈希函数 f 发生碰撞的概率不超过 $\frac{n(n-1)(l-1)}{2p}$。

在 OI 中 p 通常在 10^9 级别，而 n, L 常常达到 10^5 级别，此时的碰撞概率已经非常大。为了避免这个问题，除了使用 10^{18} 级别的模数以外，更常见的做法是**双哈希**，即使用两个独立的哈希函数 f_x, f_y。此时，两个串的碰撞概率降低至 $(l/p)^2$。

1.2　子串哈希

我们的哈希函数可以方便地提取 S 的某个子串 $S[l,r]$ 的哈希值：

$$\begin{aligned} f(S[l,r]) &= \left(\sum_{i=l}^{r} S[i]\cdot x^{r-i}\right) \bmod p \\ &= \left(\sum_{i=1}^{r} S[i]\cdot x^{r-i} - x^{r-l+1}\sum_{i=1}^{l-1} S[i]\cdot x^{l-1-i}\right) \bmod p \end{aligned}$$

于是我们只要预处理所有的前缀哈希值以及 x 的幂，就可以在 $O(1)$ 时间内计算任意子串的哈希值。

对于字符串有修改的情况，可以使用数据结构维护。

1.3　应用

以下是字符串哈希的常见应用。假设哈希值均已预处理完成。

- 比较两个字符串是否相同
- $O(\log n)$ 查询两个字符串的最长公共前缀/后缀
- $O(\log n)$ 比较两个字符串的字典序
- $O(1)$ 判断字符串是否回文
- $O(n\log n)$ 计算一个字符串的最长回文子串
- $O(n\log n)$ 查询两个串的最长公共子串，n 是两串长度的较小值

直接比较两个字符串的哈希值即可判断它们是否相等。也可以进一步用于比较两个字符串的字典序：二分出最长的相同前缀，然后比较下一位即可。

2 Border，周期与字符串匹配

2.1 Border 与周期

如果 $0 \le i < |s|$ 满足 $s[1,i] = s[|s|-i+1,|s|]$，则称 $s[1,i]$ 是 s 的一个 **border**。我们把 i 也称为 s 的一个 border。如果 $s[i+p] = s[i]$ 对所有 $1 \le i \le |s|-p$ 成立，则称 p 是 s 的一个 **周期**。如果周期 p 还满足 $|s| \bmod p = 0$，则称 p 是 s 的一个**整周期**。

下面的事实说明了 border 与周期之间的对应关系：

事实 2.1. p 是周期当且仅当 $s[1,|s|-p]$ 是 border。

因此，研究 border 与研究周期都能得到**两者**的性质。

Border 具有以下的性质：

引理 2.2. 如果 $s[1,i]$ 和 $s[1,j]$ 都是 s 的 border 且 $i < j$，那么 $s[1,i]$ 也是 $s[1,j]$ 的 border。

因此，我们通常关注 s 的最长 border。设 $f(i)$ 为 $s[1,i]$ 的最长 border 长度，则 s 的所有 border 即为 $f(n), f(f(n)), f(f(f(n))), \ldots$（$n = |s|$）。

KMP 算法可以线性求出一个字符串所有前缀的最长 border，其核心是以下引理：

引理 2.3. 若 $s[1,j]$ 是 $s[1,i]$ 的 border，则 $s[1,j-1]$ 是 $s[1,i-1]$ 的 border。反过来，若 $s[1,j-1]$ 是 $s[1,i-1]$ 的 border 且 $s[j] = s[i]$，则 $s[1,j]$ 是 $s[1,i]$ 的 border。

推论 2.4. 设 $B(i)$ 为 $s[1,i]$ 的所有 border 长度构成的集合，则

$$B(i) = \{j+1 : j \in B(i-1), s[j+1] = s[i]\} \uplus \{0\}.$$

算法 2.5. KMP 算法. 设 $1, \ldots, i-1$ 的 f 均已求出。为了求出 f_i，从长到短逐个遍历 $s[1,i-1]$ 的所有 border j，判断 $s[j+1] = s[i]$ 是否成立，首次成立时有 $f_i = j+1$。

```
f[1]=0;
for (int i=2, j=0; i<=n; i++) {
    while (j>0 && s[j+1]!=s[i]) j=f[j];
    if (s[j+1]==s[i]) j++;
    f[i]=j;
}
```

由于 j 总共只会增加 n 次，这个过程的总复杂度为 $O(n)$。

连边所有的 $f(i) \to i$ 可以形成一棵以 0 为根的有根树，一个前缀 $s[1,i]$ 的所有 border 即为 i 在这棵树上的所有祖先。

评注. 在其他资料中，f 数组也被称为 *fail* 数组/*next* 数组/失配数组。这棵树也称为 *fail* 树/失配树。

练习 2.6. P5829. 求两个前缀的最长公共 border。

练习 2.7. P2375. 求出每个前缀 $s[1,i]$ 最长的长度不超过 $i/2$ 的 border。

练习 2.8. 求出每个前缀 $s[1,i]$ 最长的长度不超过 l_i 的 border。

推论 2.4 也同时给出了维护每个 i 的 border 集合 $B(i)$ 的方法：从 $B(i-1)$ 中删去所有 $s[j+1]\neq s[i]$ 的 j，然后让所有元素加一，最后加入 0 即可。为了快速删除 $s[j+1]\neq s[i]$ 的 j，我们可以对 border 链进行划分。称满足 $s_{j+1}=s_{f_j+1}$ 的边 $f_j\to j$ 为**同色边**，同色边构成的链称为**同色链**。不难发现一条同色链上的点 j 的 s_{j+1} 都是相同的，因此它们要么都被删除，要么都不被删除。此外，若一条同色链不被删除，则这条链上方的同色链必然被删除。记 tp_j 为 j 所在的同色链链顶结点，我们可以在 KMP 过程中顺便计算出 tp_j。每次寻找需要删除的元素时，若当前的 j 需要被删除则删除 j 并令 $j\leftarrow f_j$，否则令 $j\leftarrow f_{\mathrm{tp}_j}$ 即可。这样，我们可以每次 $O(1)$ 地找到一个需要删除的 j。由于递推 $B(i)$ 时每次只会向集合中添加一个元素 0，故总共添加的元素数量不超过 n，从而总共删除的元素数量也不超过 n，复杂度 $O(n)$。

练习 2.9. CF1286E. 维护字符串 S 和序列 a，定义 i 的权值为 a 的后缀 min 在 $i+1$ 处的值。支持在 S 末尾添加一个字符同时在 a 末尾添加一个数，然后查询 S 的所有周期的权值之和。强制在线。

接下来，我们研究周期的一些性质。首先，周期可以诱导更长的周期：

事实 2.10. 如果 p,q 都是 s 的周期，那么 $p+q$ 也是 s 的周期。

推论 2.11. 如果 p 是 s 的周期，那么 kp 也是 s 的周期。

下面的两个引理则说明了周期如何诱导更短的周期：

引理 2.12. 弱周期引理. 如果 p,q 都是 s 的周期且 $p+q\le|s|$，那么 $\gcd(p,q)$ 也是 s 的周期。

证明. 不妨假设 $q>p$，我们只需证明 $q-p$ 也是 s 的周期。对任何 $i\in(p,n-(q-p)]$，都有 $s_{i+q-p}=s_{i-p}=s_i$；对任何 $i\le p$，都有 $s_{i+q-p}=s_{i+q}=s_i$。这就完成了证明。 □

引理 2.13. 周期引理. 如果 p,q 都是 s 的周期且 $p+q-\gcd(p,q)\le|s|$，那么 $\gcd(p,q)$ 也是 s 的周期。

证明. 先考虑 p,q 互质的情况，此时我们需要说明当 $p+q-1\le|s|$ 时 s 只由一种字符构成。令字符串下标从 0 开始，则对于任何 $0\le i<p-1$，都有 $s_i=s_{(i+q)\bmod p}$。考虑连边所有的 $(i,(i+q)\bmod p)$，则一个连通块内的点 i 对应的 s_i 都是相同的。如果允许 $i=p-1$，那么这就构成 $0,1,\ldots,p-1$ 的一个环，而删掉 $i=p-1$ 这条边后图仍然连通。于是 s 只由一种字符构成。

对于 p,q 不互质的情况，只需要把字符串下标按照模 $\gcd(p,q)$ 分类即可。 □

周期引理直接刻画了所有的短周期。

推论 2.14. 设 p 是 s 的最短周期，$q \le |s|-p+1$，则 q 是周期当且仅当 $p|q$。

推论 2.15. s 的不超过 $|s|/2$ 的所有周期构成一个等差数列，s 的长度 $\ge |s|/2$ 的所有 border 构成一个等差数列，且公差均为 s 的最短周期。

推论 2.16. s 的非空 border 可以划分为至多 $\lfloor \log_2 |s| \rfloor$ 个等差数列，因此周期也对应划分为至多 $\lfloor \log_2 |s| \rfloor$ 个等差数列。

评注. 可以这样构造一个达到 $\frac{\log_2|s|}{2}$ 的字符串：设 c_i 是两两不同的字符，令 $s_1=c_1$，$s_i=s_{i-1}c_is_{i-1}$ 即可。

我们可以将周期的这些性质与失配树联系起来。

引理 2.17. 设 $p=i-f_i$，则当 $\mathrm{tp}_i \ne i$ 时有 $f_{\mathrm{tp}_i} < p-1$。

证明. 反证，假设 $f_{\mathrm{tp}_i} \ge p-1$。首先，$p$ 是 $s[1,i]$ 的最短周期。由于 tp_i 和 f_{tp_i} 都是 $s[1,i]$ 的 border，故 $i-\mathrm{tp}_i$ 和 $q=i-f_{\mathrm{tp}_i}$ 都是 $s[1,i]$ 的周期。由于 $f_{\mathrm{tp}_i} \ge p-1$，故 $q+p-1 \le i$，从而推论 2.14 说明 i 到 f_{tp_i} 这条链刚好是一个等差数列 $\{i-kp\}$，故 $f_{\mathrm{tp}_i}=\mathrm{tp}_i-p$。

由于 p 是 $s[1,i]$ 的周期，有 $s_{\mathrm{tp}_i+1}=s_{tp_i+1-p}=s_{f_{\mathrm{tp}_i}+1}$，故 $f_{\mathrm{tp}_i} \to \mathrm{tp}_i$ 是同色边，与 tp_i 的定义矛盾。 □

推论 2.18. i 到根的路径上至多有 $O(\log i)$ 条异色边。

证明. 令 $p=i-f_i$。若 $p \ge i/3$，则 $f_i \le 2i/3$，跳一条边后 $i' \le 2i/3$。若 $p < i/3$ 且 $\mathrm{tp}_i \ne i$，则 $f_{\mathrm{tp}_i} < p-1 < i/3$，故跳到链顶再跳一条异色边后 $i' \le i/3$。最后，若 $p<i/3$ 且 $\mathrm{tp}_i=i$，则必有 $f_{f_i}=i-2p$，$s_{f_i+1}=s_{f_{f_i}+1}$，故 $\mathrm{tp}_{f_i} \ne f_i$，从而跳两条异色边后 $i' \le i/3$。 □

练习 2.19. 在预处理出 f 后，$O(\log n)$ 地计算某个前缀 $s[1,i]$ 的所有 border 的等差数列表示。

练习 2.20. 设计一个严格 $O(\log n)$ 在末尾添加字符并计算 f 的算法，从而支持在 Trie 上计算 f。

练习 2.21. P5287. 支持在字符串末尾添加若干个相同字符、恢复到某个版本、查询所有 $f(i)$ 的和。

解答. 考虑所有的极长同色段。一个 border 除了开头结尾两段以外，中间的段必须满足字符和数量都相同。因此，考虑将每个极长同色段缩成一个字符 (c,l)，然后维护这个字符串上的 border，其中首段的判定条件需要稍加修改。

容易验证弱周期引理仍然成立，因此使用严格 $O(\log n)$ 添加字符的算法。不完整结尾段的 f 可以在计算完整结尾段的 f 的过程中顺便计算。

另解. 使用可持久化线段树直接维护 KMP 自动机。

练习 2.22. P4156. 令 P 为字符串 s 的所有周期构成的集合，询问 $[1,w]$ 内有多少个数可以被 P 中元素线性表示，即询问 $[1,w]$ 内有多少个数可以表示为 $\sum_{p\in P} k_p p$，其中 $k_p \geq 0$。

解答. 假设最短周期为 p,考虑同余背包（或者说同余最短路）。由于周期可以被划分为 $O(\log n)$ 个等差数列，可以逐个等差数列考虑。

对于一个等差数列 $\{x+id : 0 \leq i < l\}$，在 $\bmod x$ 下做同余背包，则 t 可以转移到所有 $(t+id) \bmod x$ $(0 \leq i < l)$，这会形成 $\gcd(x,d)$ 个环，一个环上每个点可以转移到其后 $l-1$ 个点。每个环可以在答案最小的元素处断开变成在序列上转移，单调队列即可。

接下来需要在不同等差数列的模数之间切换。假设之前的模数是 x_1，需要切换到模数 x_2 上。对每个 $0 \leq t < x_1$，其可以转移到所有的 $(t+ix_1) \bmod x_2$，和前面类似地做即可。

相比周期，整周期要求的整除性质使它更容易计算。

引理 2.23. 如果 s 存在整周期 $d \neq |s|$，则所有整周期都是最短周期的倍数。这也说明此时最短周期也是整周期。

这种特征允许我们使用**试除法**快速计算最小整周期。

引理 2.24. 假设关于正整数 $x \leq M$ 的命题 $P(x)$ 具有如下性质：

- 如果 $P(x)$ 成立，则 $P(kx)$ 对所有 $k \geq 1$ 都成立。
- 如果 $P(a), P(b)$ 都成立，则 $P(\gcd(a,b))$ 也成立。

那么如果存在 x_0 使得 $P(x_0)$ 成立，则存在唯一的整数 d 使得 $P(x)$ 成立当且仅当 x 是 d 的倍数，且以下算法可以求出 d：初始令 $x = x_0$，逐个枚举 x_0 的质因子 p，若 $P(x/p)$ 成立则令 $x = x/p$，并重复此过程。最终得到的 x 即为所求的 d。此算法只需要 $O(\log x_0)$ 次判定 $P(x)$。

评注. 这也给出了在已知某个长度不超过 $|s|/2$ 的周期的情况下计算最短周期的一种方法。除了整周期和短周期以外，阶也是常见的满足此性质的量。

令 $P(x)$ 表示 x 是一个整周期，则只需要 $O(\log |s|)$ 次判断 $|s|-x$ 是否是 border，这容易通过各种方式完成。

练习 2.25. P3538. 多次询问子串最短整周期。

2.2 字符串匹配

字符串匹配指的是在字符串 t（文本串）中寻找另一个字符串 s（模式串）的出现位置。

KMP 算法利用 s 的所有 border 计算下列形式的匹配：对每个 $1 \leq i \leq |t|$，计算 $l(i) = \max\{l : s[1,l] = t[i-l+1,i]\}$。注意到如果使用一个特殊字符 # 将 s 和 t 连接起来，则 $l(i)$

就是 $s\#t[1,i]$ 的最长 border 长度。于是我们可以直接使用 KMP 算法计算出所有的 $l(i)$。不过，我们实际上并不需要真的做这样的连接。

```
// |s|=n, |t|=m
for (int i=1, j=0; i<=m; i++) {
    while (j>0 && s[j+1]!=t[i]) j=f[j];
    if (s[j+1]==t[i]) j++;
    l[i]=j;
    if(j==n) {
        // 找到一次出现
        j=f[j];
    }
}
```

我们还可以利用 l 数组表示出 s 的**所有前缀**在 t 中的出现情况。

引理 2.26. $s[1,j]$ 出现在了 $t[i-j+1,i]$ 当且仅当 $s[1,j]$ 是 $s[1,l[i]]$ 的一个 border（这里允许等于自身）。

因此，在 t 中以 i 为结尾的所有 s 的前缀即为 $l[i]$ 在 fail 树上到根的链。反过来，如果对每个 i 都在 $l[i]$ 处放置一个标记 i，则 $s[1,j]$ 的所有出现位置即为 fail 树上 j 的子树内的所有标记。

练习 2.27. P3426. 给定字符串 t，求一个最短的字符串 s，使得对于每一个 $1\le i\le |t|$，都存在一次 s 在 t 中的出现 $[l,r]$ 使得 $i\in[l,r]$。

如果一个串 s 的两次出现 $[i,i+|s|)$ 和 $[i+d,i+d+|s|)$ 之间的距离 $d\le|s|$，那么这两次出现对应的子串 $[i,i+d+|s|)$ 具有一些良好的性质。

事实 2.28. 这两次出现的重叠部分必然是 s 的一个 border。d 是 $[i,i+d+|s|)$ 的一个周期。

引理 2.29. 假设 $t[i,i+|s|)=t[i+d,i+d+|s|)=s$ 且 $p\le d\le|s|$。以下三条等价：

1. $t[i,i+d+|s|)$ 有周期 p
2. $t[i+p,i+p+|s|)$ 是 s 的一次出现
3. $\gcd(p,d)$ 是 s 的周期。

证明. (a)⇒(b)：由于 p 是周期，将 $[i,i+|s|)$ 右移 p 得到的还是相同的字符串，即 $t[i+p,i+p+|s|)=t[i,i+|s|)=s$。

(b)⇒(c)：p 是子串 $[i,i+p+|s|)$ 的一个周期。另一方面，d 也是这个子串的一个周期，从而周期引理说明 $\gcd(p,d)$ 也是它的一个周期，进而是 s 的一个周期。

(c)⇒(a)：只需说明 $g=\gcd(p,d)$ 是 $[i,i+d+|s|)$ 的周期。考虑模 g 同余的两个位置 x,y。如果 x,y 均位于 $[i,i+|s|)$ 或均位于 $[i+d,i+d+|s|)$ 内，那么 g 是 s 的周期即可说明

$t[x]=t[y]$。因此只需考虑 $x\in[i,i+d),y\in[i+|s|,i+d+|s|)$ 的情况。由于 d 是周期，故 $t[y]=t[y-d]$，而 $y-d\le i+|s|$ 且 $y-d\equiv x\pmod g$，故 $t[x]=t[y-d]=t[y]$。 □

推论 2.30. 若 $2|s|\ge|t|$，则 s 在 t 中的所有出现位置构成一个等差数列。如果出现了至少三次，那么这个等差数列的公差为 s 的最短周期。

证明. 只需要考虑 s 至少出现三次的情况。考虑 s 的任意三次相邻出现位置 $i,i+d_1,i+d_1+d_2$，那么 d_1 和 d_2 均为 s 的周期，且 s 的这三次出现在 t 中形成了一个长度为 $d_1+d_2+|s|$ 的子串，故 $d_1+d_2+|s|\le|t|$。由于 $|t|\le 2|s|$，就有 $d_1+d_2\le|s|$。设 p 是 s 的最小周期，则 $p+d_1\le|s|,p+d_2\le|s|$，故推论 2.14 说明 d_1,d_2 均为 p 的倍数。于是引理 2.29 说明 $[i+p,i+p+|s|)$ 和 $[i+d_1+p,i+d_1+p+|s|)$ 均为 s 的出现。而 $i,i+d_1,i+d_1+d_2$ 是相邻的出现，故必有 $d_1=d_2=p$。 □

例 2.31. 基本子串字典. 多次询问子串的最长 border。

2.3 Z 函数

KMP 算法计算了每个前缀的最长 border 长度。我们也可以计算一个“对称”的量 $z(i)=\mathrm{LCP}(s[i,|s|],s)=\max\{l:s[i,i+l-1]=s[1,l]\}$，这就是 Z 函数。以下算法可以在线性时间内计算 Z 函数：

算法 2.32. Z 算法/扩展 KMP. 从左往右扫描 $i\ge 2$ 并维护最大的 $i+z(i)$。假设目前的最值在 j 处取到。当 $i-1$ 移动到 i 时，由于 $s[j,j+z(j)-1]=s[1,z(j)]$，就有 $s[i,j+z(j)-1]=s[1+i-j,z(j)]$，从而 $z(i)\ge\min(j+z(j)-i,z(1+i-j))$。从这个下界开始暴力尝试扩展即可。

注意到只有 $z(1+i-j)\ge j+z(j)-i$ 时才有可能发生扩展，此时 $z(i)$ 的初值为 $j+z(j)-i$，于是每次扩展都会增大 $i+z(i)$ 的最大值。而 $i+z(i)\le n+1$，于是这个过程中总共至多扩展 n 次。

```
// n>=2
z[1]=n; z[2]=0;
for (int i=2, j=2; i<=n; i++)
{
    z[i]=max(0, min(j+z[j]-i, z[1+i-j]));
    while (i+z[i]<=n && s[z[i]+1]==s[i+z[i]]) z[i]++;
    if (i+z[i]>j+z[j]) j=i;
}
```

可以使用 z 数组计算 f 数组：对于 $j\in[i,i+z(i)-1]$，有 $f(j)\ge j-i+1$。而随着 i 的增加，$j-i+1$ 只会越来越小。因此可以从小到大枚举 i，然后从大到小枚举 $j\in[i,i+z(i)-1]$，将对应位置的 $f(j)$ 设为 $j-i+1$，直到遇见一个以前被赋值过的位置为止。

练习 2.33. [NOIP2020] 字符串匹配. 给定字符串 s，计算有多少种方案可以将其拆成 $(AB)^iC$ 的形式，要求 A,B,C 非空、$i \geq 1$ 且 $F(A) < F(C)$，其中 $F(S)$ 表示 S 中出现奇数次的字符数量。

解答. 枚举 AB 的长度 l，容易用 Z 函数计算出可行的最大 i。这些 C 对应的 $F(C)$ 只有至多两种，其中一种为 $F(s[l+1:])$，另一种为 $F(s)$。对于前者，每次 l 移动时 $F(s[l+1:])$ 只会变化 1，而后者始终保持不变，这样可以直接 $O(1)$ 维护合法的 A 的数量。

3 有限状态自动机

3.1 基本概念

定义 3.1. 语言. 语言是一个字符串的集合。

例 3.2. 以下是一些语言的例子：

- $\{a^n : n \geq 0\}$：所有由 a 组成的字符串组成的集合
- $\{a^n b^n : n \geq 0\}$：所有形如 $a^n b^n$ 的字符串组成的集合
- $\{ww^R : w \in \Sigma^*\}$
- $\{a^n b^n c^n : n \geq 0\}$
- $\{\langle M, x\rangle :$ C++ 程序M 在输入 x 上运行时会停机$\}$

有限状态自动机是描述状态关于输入变化的数学模型。它的一个应用是识别语言：对于一个语言 L，我们希望设计一个自动机 M，使得对于任意字符串 s，M 能够判断是否 $s \in L$。

定义 3.3. 确定性有限状态自动机（DFA）. 有限状态自动机是一个五元组 $(Q, \Sigma, \delta, q_0, F)$，其中：

- Q 是有限的状态集合；
- Σ 是有限的输入符号集合（字符集）；
- $\delta : Q \times \Sigma \to Q$ 是状态转移函数。$\delta(q, a)$ 表示在状态 q 下读入符号 a 时转移到的新状态；
- $q_0 \in Q$ 是初始状态；
- $F \subseteq Q$ 是接受状态集合。

以下是字符串 s 在 DFA M 上的处理过程：

```
int state = q0;
for (char c : s) {
    state = delta(state, c);
}
if (state in F) {
    // s 被接受
} else {
    // s 被拒绝
}
```

记 $L(M)$ 为自动机 M 接受的语言，即所有被 M 接受的字符串组成的集合。

本章中提到的自动机都指**有限状态自动机**。DFA 可以用有向图表示。

容易发现在 DFA 上识别字符串的复杂度是 $O(|s|)$ 的。

练习 3.4. 设计 DFA，识别以下语言：

- $\{(ab)^n : n \geq 0\}$。
- 出现过偶数次 01 的字符串组成的集合。
- 出现过 00，但没有出现过 111 的字符串组成的集合。

练习 3.5. KMP 自动机. 给定字符串 s，设计 DFA，识别：

- 以 s 为前缀的字符串组成的集合
- 以 s 为后缀的字符串组成的集合
- 包含 s 作为子串的字符串组成的集合

可以被 DFA 识别的语言称为**正则语言**。

如果允许一次转移到多个不同的状态，则称为**非确定性有限状态自动机（NFA）**。

定义 3.6. 非确定性有限状态自动机（NFA）. 非确定性有限状态自动机是一个五元组 $(Q, \Sigma, \delta, q_0, F)$，其中：

- Q 是有限的状态集合；
- Σ 是有限的输入符号集合（字符集）；
- $\delta : Q \times \Sigma \to 2^Q$ 是状态转移函数。$\delta(q, a)$ 表示在状态 q 下读入符号 a 时可以转移到的新状态集合；
- $q_0 \in Q$ 是初始状态；
- $F \subseteq Q$ 是接受状态集合。

如果存在一列状态 $q_1, \ldots, q_n$，使得 $q_n \in F$，且对于每个 $i = 1, \ldots, n$ 都有 $q_i \in \delta(q_{i-1}, s_i)$，则称字符串 s 被 NFA 接受。换句话说，NFA 可以“猜”出一条路径，使得读入字符串 s 后到达接受状态。

评注. *一些定义使用字符集 $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$，其中 ϵ 为空字符。换句话说，允许在不输入字符的情况下转移。不难将这样定义的 NFA 转为不使用 ϵ 转移的 NFA。*

在 NFA 上识别字符串可以简单地做到 $O(|s||Q|^2/w)$ 或者 $O(\sum_i s_i$ 涉及的转移边数$)$。

定理 3.7. NFA 和 DFA 的能力相同，即对于任意 NFA M，存在一个 DFA M'，使得 $L(M) = L(M')$，反之亦然。换句话说，L 是正则语言当且仅当它能被一个 NFA 识别。

证明. 显然 DFA 是一个 NFA。

对于一个 NFA $M = (Q, \Sigma, \delta, q_0, F)$，可以构造一个状态集合为 $Q' = 2^Q$ 的 DFA，表示目前可以到达哪些状态。 □

另一种描述正则语言的方式是**正则表达式**。

定义 3.8. 正则表达式. 正则表达式是由以下规则确定的语言：

- 空表达式是正则表达式，表示语言 $\varnothing$；
- 任意字符 $c \in \Sigma \cup \{\epsilon\}$ 都是一个正则表达式，表示语言 $\{c\}$；
- 如果 R_1, R_2 是正则表达式，则 $R_1|R_2$（也记作 $R_1 \cup R_2$）也是正则表达式，表示语言

$L(R_1) \cup L(R_2)$；
- 如果 R_1, R_2 是正则表达式，则 R_1R_2 也是正则表达式，表示语言 $\{w_1w_2 : w_1 \in L(R_1), w_2 \in L(R_2)\}$；
- 如果 R_1 是正则表达式，则 R_1^* 也是正则表达式，表示语言 $\{w_1w_2\cdots w_n : n \geq 0, w_i \in L(R_1)\}$。

评注. 在工程实践中，正则表达式通常还包括一些扩展的语法，例如：
- $R+$ 表示 RR^*，即至少出现一次 R；
- $R?$ 表示 $R|\epsilon$，即出现 *0* 次或 *1* 次 R；
- $R\{n, m\}$ 表示 R 出现 n 到 m 次；
- $[abc]$ 表示 $a|b|c$，即出现一个字符 a, b, c 中的任意一个；
- `[^abc]` 表示出现非 a, b, c 的任意一个字符；
- $[a-z]$ 表示出现 a 到 z 之间的任意一个字母；
- `.` 表示任意字符。

定理 3.9. L 是正则语言当且仅当它可以由一个正则表达式描述。

证明. 给定一个正则表达式，容易将其转为一个 NFA。

反过来，给定一个 DFA，我们需要将其转为正则表达式。我们考虑使用正则表达式增强的 DFA：每个转移增强为 $\delta(q, R)$，接收一段被正则表达式 R 匹配的字符串来转移。然后考虑逐个去除每个状态 q，这可以通过将所有 $u \to q \to^* q \to v$ 的转移直接利用正则表达式缩成 $u \to v$ 来完成。 □

正则表达式转成的 NFA 具有 $O(|R|)$ 的边数，因此用正则表达式识别字符串可以做到 $O(|s||R|)$。

利用这些描述正则语言的工具，我们可以给出一些正则语言封闭性的结论：

定理 3.10.
- 正则语言的交、差、并、补都是正则语言
- 正则语言的反转也是正则语言
- 正则语言的同态像 $h(L) = \{h(w) : w \in L\}$ 也是正则语言（同态是满足 $h(s_1s_2) = h(s_1)h(s_2)$ 的函数）
- 正则语言的逆同态像 $h^{-1}(L) = \{w : h(w) \in L\}$ 也是正则语言

3.2　等价类

定义 3.11. 对于一个语言 L，如果字符串 x, y 满足对任何字符串 z 都有 $xz \in L \iff yz \in L$，则称 x, y 在 L 下等价，记作 $x \equiv_L y$。

一个等价类可以视为自动机上的同一个结点，这就给出了如下定理。

定理 3.12. Myhill-Nerode 定理. 语言 L 是正则语言当且仅当 L 在 $\equiv_L$ 下的等价类数量有限，且该数量是最小 DFA 的状态数。

证明. 首先假设 L 是正则语言，按照字符串在 DFA 上移动到的状态来划分等价类，此等价类划分必然是 $\equiv_L$ 等价类划分的一个加细。

反过来，可以根据 $\equiv_L$ 的等价类构造 DFA。每个等价类都是 DFA 的一个状态。每个等价类选择其中最短的任意一个字符串作为代表元。记 $\bar{x}$ 为字符串 x 所在的等价类。对于代表元为 s 的等价类 $\bar{s}$，其转移为 $\delta(\bar{s}, c) = \overline{sc}$。一个等价类为终止状态当且仅当其代表元在 L 中。我们说明这个 DFA 可以正确识别 L。只需要归纳证明串 s 的任何前缀 $s[1,i]$ 在 DFA 上走到的等价类都是 $\overline{s[1,i]}$。$i = 0$ 时显然成立；假设对 i 成立，则由于 $s[1,i] \equiv_L \overline{s[1,i]}$，有 $s[1,i+1] \equiv_L \overline{s[1,i]}s[i+1]$，而后者所在的等价类依定义即为 $s[1,i+1]$ 走到的结点，于是对 $i+1$ 也成立。 □

可以利用这个想法来**最小化 DFA**，即找到一个状态数最小的识别相同语言的 DFA。我们先定义限制长度的等价关系：

定义 3.13. 对于一个语言 L，如果字符串 x, y 满足对任何长度不超过 k 的字符串 z 都有 $xz \in L \iff yz \in L$，则称 x, y 关于 (L, k) 等价，记作 $x \equiv_L^k y$。

下面给出了关于此等价关系的一些基本事实。

事实 3.14.

- 关于 $(L, 0)$ 的等价类为 $\{L, \bar{L}\}$，在自动机上体现为 $\{F, Q\backslash F\}$。
- 如果 $x \equiv_L^{k+1} y$，那么有 $x \equiv_L^k y$。因此，随着 k 的增加，等价类会不断加细。
- $x \equiv_L^{k+1} y$ 当且仅当对任何 $c \in \Sigma \cup \{\epsilon\}$ 都有 $xc \equiv_L^k yc$。
- $\equiv_L^k$ 导出的 DFA 可以正确判定长度不超过 k 的串是否属于 L。

引理 3.15. 如果 (L, k) 的等价类数量和 $(L, k+1)$ 的等价类数量相同，那么这就是关于 L 的等价类。

证明. 我们先说明这就是关于 $(L, k+2)$ 的等价类，从而递推说明对任何 $K \geq k$ 都是关于任何 (L, K) 的等价类。我们有 $x \equiv_L^{k+2} y$ 当且仅当对任何 $c \in \Sigma \cup \{\epsilon\}$ 都有 $xc \equiv_L^{k+1} yc$。由于 (L, k) 和 $(L, k+1)$ 的等价类相同，这等价于 $\forall c \in \Sigma \cup \{\epsilon\}, xc \equiv_L^k yc$，进而等价于 $x \equiv_L^{k+1} y$。

现在，考虑同一个等价类内的两个字符串 x, y。对任何字符串 z，都有 $x \equiv_L^{|z|} y$，从而 $xz \in L \iff yz \in L$。这就说明 $x \equiv_L y$。 □

这就给出了 DFA 最小化的一个直接算法。

算法 3.16. DFA 最小化. 初始令等价类划分为 $\{F, Q\backslash F\}$。重复以下过程：

```
1        for (c in Sigma)
2            for (等价类 C)
3            {
```

```
                for (状态 q in C)
                    记录 delta(q, c) 所在的等价类;
                根据记录加细 C
            }
        如果没有发生加细，停止。
```

设 $|Q|=n$。由于加细至多进行 n 次，故以上过程至多执行 n 轮；每轮中每个结点和字符会被枚举一次，故每轮的复杂度为 $O(|\Sigma|n)$。因此总复杂度为 $O(|\Sigma|n^2)$。

Hopcroft 算法可以在 $O(|\Sigma|n\log n)$ 的时间内计算 DFA 最小化。相比枚举等价类进行加细，该算法的想法则是枚举等价类然后加细所有**能被此等价类加细**的等价类。

算法 3.17. Hopcroft 算法. 维护一个等价类集合 $\mathcal{C}$ 和待用于加细的等价类集合 $\mathcal{A}\subseteq\mathcal{C}$（称为“证据”集合）。使用证据 A 和字符 c 加细 $\mathcal{C}$ 的过程为：对每个 $C\in\mathcal{C}$，将其划分为 U,V，使得 $\delta(U,c)\subseteq A$，$\delta(V,c)\subseteq\bar{A}$，并丢弃空的那个（如果有）。

每次从 $\mathcal{A}$ 中取出一个等价类 A 并删除。枚举字符 c，用 A,c 加细 $\mathcal{C}$。加细完成后，A 将不再可能加细未来产生的等价类。在加细过程中对 $\mathcal{A}$ 进行维护。假设某个等价类 C 分裂为非空的两部分 U,V，如果 $C\in\mathcal{A}$，那么可以将 C 替换为 U 和 V；否则，说明 C 已经永远无法加细未来的 $\mathcal{C}$，此时只需要向 $\mathcal{A}$ 中加入 U,V 中的**任意一个**，我们加入其中较小的那个。这是因为 $\delta(q,c')$ 只可能转移到 $\bar{C}$、U、V 中的其中一个，我们使用其中两个就能完成正确的划分。

```
按照 {F,Q-F} 划分等价类;
sA={F};
while (sA not empty)
{
    取出一个 A in sA 并删除之;
    for (c in Sigma)
    {
        S = {q : delta(q, c) in A};
        for (和 S 有交的等价类 C)
        {
            令 U = S cap C, 如果 U=C 则跳过
            将 U 从 C 中分裂出去
            如果 |U|>|C| 则交换 U 和 C
            将 U 加入 sA
        }
    }
}
```

注意分裂需要 $O(|U|)$ 的实现，这可以惰性完成。用不同的编号标记不同等价类，并记录每个状态 q 所属的等价类，在遍历某个等价类中的元素时据此确认其是否真的属于此等价类。总共的代价不超过过程中向 sA 中加入的所有等价类的入度之和的常数倍。对于每个状态 $q\in Q$，整个过程中包含它且被加入到 sA 中的集合只有 $\log n$ 个。每条边 $\delta(q,c)$ 只会被访

问 $\log n$ 次，总复杂度 $O(|\Sigma| n \log n)$。

评注. 这里是一个 $O(m \log n)$ 复杂度的算法。

对于某些问题，我们可以由较短的串诱导的等价类逐步得到实际的等价类。

例 3.18. 给定一个 $\{0,1\}^3 \to \{0,1\}$ 的函数 f 和一个 01 串 s，每次可以把 s 中连续的三个字符替换为 f 的输出。问是否可以把 s 变为 1。

解答. 令 L 为所有可以变成 1 的 s 构成的语言。考虑 $\equiv_L$ 在所有长度不超过 k 的串上给出的等价类划分，这个划分对应的 DFA M_k 可以正确判定长度不超过 k 的串是否属于 L。另一方面，我们容易通过枚举第一次替换的位置来将其改造为一个正确判定长度不超过 $k+2$ 的串的 NFA，再将其转为 DFA，且这个过程与 k 无关。因此，如果这样转换后得到的 DFA 与 M_k 等价，我们就有 $L(M_k) = L$。

但是我们无法真的判断两个字符串是否在 $\equiv_L$ 下等价，我们可以将其用某个 $\equiv_{L,l}$ 替代。只要 $l \geq k$，$\equiv_{L,l}$ 导出的 DFA 就能正确判定长度不超过 k 的串。也就是说，我们可以这样枚举：从小到大枚举 k，在 $\equiv_{L,k}$ 下计算长度不超过 k 的串的等价类划分，并导出 DFA M_k，判定 M_k 转换出的能够判定长度不超过 $k+2$ 的串的 DFA 是否与 M_k 等价，若等价则此 DFA 即为所求。

练习 3.19. 证明如果 L 是正则语言，那么此算法必会终止。

评注. 这个问题稍微变化一些就无法用这个方法解决了。例如，如果字符集增大到 4，那么存在一个 f 使得 L 不是正则语言，而是**上下文无关语言**（*CFL*），此时只能用区间 *dp* 来做。

实际上，这个问题相当于给出了一组替换规则 $0 \to s, 1 \to t$，这是一种**上下文无关文法**（***CFG***）。一般来说，*CFG* 的正则性是**不可判定**的。

3.3　在自动机上 DP

自动机给出了一个判定字符串的方法，我们可以利用自动机做一些计数或者最优化问题。比如说，计算语言 L 中有多少个长度不超过 n 的字符串 t，这个相当于在自动机上走不超过 n 步后到达一个接受状态的方案数；计算 L 中最短的字符串，这个相当于在自动机上跑最短路。

例 3.20. 禁止子串计数. 给定字符串 s，求有多少个长为 n 的字符串 t 不包含 s 作为子串。

例 3.21. P15650 [省选联考 2026. 摩卡串] 给定字符串 s，求一个长度最短的字符串 t，使得

- s 是 t 的子串；
- t 恰好有 k 个子串的字典序小于 s。

解答. 先考虑判断一个字符串的字典序是否小于 s 的自动机。显然，这个自动机是这样设计的：首先有状态 $0, 1, \ldots, |s|$，其中状态 i 表示已经匹配了 $s[1, i]$。初始状态为 0，对于每个状

态 i，如果读入的字符小于 $s[i+1]$，则转移到一个“已经小于”的状态；如果读入的字符等于 $s[i+1]$，则转移到状态 $i+1$；如果读入的字符大于 $s[i+1]$，则转移到一个“已经大于”的状态。那么容易发现，在这个自动机上处于悬而未决的状态当且仅当当前的串是 s 的一个前缀。

我们可以利用这一点来设计计数字典序小于 s 的子串的自动机。假如我们对 t 的每一个位置 i 都维护 $t[i,|t|]$ 在上面的自动机上的状态，那么状态可以分为三类：

- 已经小于 s 的状态，以后也永远小于 s
- 已经大于 s 的状态，以后也永远大于 s
- 处于未决定状态的那些 i，即满足 $t[i,|t|]$ 是 s 的一个前缀的 i

关键在于未决定状态的那些 i。根据 border 的道理，如果我们知道最长的一个这样的 $t[i,|t|]$（也就是 i 最小的那个），那么其他的 $t[i',|t|]$ 就是它的所有 border。因此，我们只需要记录 t 在 KMP 自动机上的状态就能知道哪些 i 处于未决定状态了。

为了计数，我们只需要知道以下这些量：

- 目前已经有多少个子串小于 s
- 有多少个 i 已经处于小于 s 的状态
- 当前在 KMP 自动机上的状态

将这些作为自动机的状态即可。实际上不需要显式构造，在状态中记录一下这几个量即可，新加入字符导致的转移容易通过预处理 KMP 自动机的一些信息完成。另外，如果第二个量已经达到了 j，那么之前必然已经产生过 $1+2+\cdots+j$ 个小于 s 的子串，因此这一维可以限制到 $O(\sqrt{k})$。

由于还有 s 是 t 的子串的限制，可以再加一维 0/1 表示是否已经出现过 s。在自动机上跑最短路即可，总复杂度 $O(nk\sqrt{k})$

3.4　DAG 链剖分

例 3.22. 给一个序列，多次询问字典序第 k 小的子序列的长度。

完整的 DAG 剖分还需要使得每个点只作为一条重边的终点，这可以通过额外计算以每个点为结尾的路径数量实现，一条边是重边当且仅当双向的“重儿子”关系成立。这样剖分可以保证每条路径上至多有 $\log S$ 条轻边，其中 S 是 DAG 上的路径数量。DAG 剖分在 SAM 或者是树套树结构中有一些应用。

4 Trie 和 AC 自动机

4.1 Trie

Trie（也称字典树）是一棵有根树，其每条边上有一个字符，且每个点的出边上的字符互不相同。Trie 上的每个结点都对应着一个字符串，即将从根到这个结点的路径上的字符顺次写下得到的字符串。反过来，一个字符串在 Trie 中对应着至多一个结点。

给定若干字符串，可以构造一棵表示所有这些字符串的 Trie。从一个空的根节点开始，依定义逐个插入字符串到 Trie 中即可。

不难发现 Trie 的祖先关系对应着字符串的前缀关系。也就是说，Trie 的每个结点对应的字符串是其所有后代结点对应的字符串的前缀。

练习 4.1. Trie 上查询. 给定若干字符串组成的语言 L 和一个字符串 s。利用 Trie 完成以下任务：

- 判断 s 是否在 L 中
- 计算 s 是 L 中多少个字符串的前缀
- 计算 L 中有多少个字符串是 s 的前缀

4.1.1 压缩 Trie

将 Trie 中除了根节点以外的只有一个孩子的结点收缩即可得到压缩 Trie。压缩 Trie 的每条边存储着一个字符串，且每个点的各个出边的字符串的**首字符**互不相同。普通 Trie 的结点数量上界为 $\sum_{S\in L}|S|$，而压缩 Trie 的结点数量不超过 $2|L|$。

练习 4.2. 压缩 Trie 的实现. 实现压缩 Trie 上的插入和查询操作。

由于每条边依然需要存储字符串，直接实现的空间复杂度与普通 Trie 相同。不过，可以利用压位技巧将空间复杂度除掉一个 $w/\log|\Sigma|$。

4.1.2 01 Trie

给定若干个非负整数，可以将其视为固定位数的 01 字符串考虑，这样得到的 Trie 称为 01 Trie。01 Trie 和值域为 $[0,2^v)$ 的线段树同构。它常常被用于处理异或相关的问题。

练习 4.3. 实现压缩 01 Trie。每条边存储子树内任意一个叶子的值以及分叉位。

练习 4.4. 维护一个非负整数集合 S，支持

- 插入/删除一个数
- 给定 x，查询 $\{x\oplus y : y\in S\}$ 的最大和最小值
- 给定 x,y，查询 $\#\{z : z\in S, x\oplus z\le y\}$
- 给定 x,k，查询 $\{z\oplus x : z\in S\}$ 的第 k 大/小值

练习 4.5. P11872. 支持单点修改、全局加一、查询全局异或和。提示

解答. 将整数视为从低位到高位的字符串，维护 01 Trie。此时，全局加一可以这样完成：

1. 初始令 $i=0$，u 为根结点
2. u 子树内的值异或 2^i，这体现为交换 u 的左右儿子，同时更新一些信息
3. 令 $i \leftarrow i+1$，$u \leftarrow \texttt{son[u][0]}$
4. 如果 u 为空，则停止；否则回到第二步

由于 01 Trie 和线段树同构，也可以像线段树一样支持分裂、合并、可持久化等等。

练习 4.6. [Ynoi2013] Ynoi. 支持区间异或一个数、区间排序、查询区间异或和。

4.2　AC 自动机

AC（Aho-Corasick）自动机是 KMP 自动机与 Trie 的结合。具体来说，AC 自动机在 Trie 的基础上增加了以下两个数组：

- `fail` 数组（后缀链接）：设 Trie 上的结点 u 表示的字符串为 $s[u]$，则 `fail[u]` 指向的结点 v 满足：$s[v]$ 是 $s[u]$ 的一个**真**后缀，且 v 是这样的结点中 $s[v]$ 最长的一个。
- 转移 $\delta(u,c)$：$\delta(u,c)$ 指向的结点 v 满足 $s[v]$ 是 $s[u]+c$ 的一个后缀（可以相等），且 v 是这样的结点中 $s[v]$ 最长的一个。

评注. *需要注意 $fail$ 和 δ 未必指向 Trie 上的祖先结点。*

`fail` 会形成一棵树。与 border 类似，有以下性质成立：

性质 4.7. 如果 $s[v]$ 是 $s[u]$ 的一个真后缀，那么 $s[v]$ 也是 $s[\texttt{fail[u]}]$ 的一个后缀。因此，$s[u]$ 的所有在 Trie 上存在的后缀即为其在 fail 树上到根的链。

可以 BFS 遍历 Trie 来计算这两个数组以构建 AC 自动机。

练习 4.8. AC 自动机的实现. 给定若干个字符串，构建其对应的 AC 自动机。

评注. *尽管 Trie 是支持在线插入的，但是 AC 自动机无法直接支持在线插入。如果需要，可以考虑二进制分组。*

如果字符集很大，还可以使用可持久化数组来维护 δ。此外，如果只需要 `fail` 数组，那么除了建立 Trie 以外复杂度可以与 $|\Sigma|$ 无关。具体地，可以这样从 $\texttt{fail}[u]$ 转移到 $\texttt{fail}[\text{son}_{u,c}]$：初始设 $v=\texttt{fail}[u]$，暴力跳 fail 找到第一个存在 $\text{son}_{v,c}$ 的结点，令 $\texttt{fail}[\text{son}_{u,c}]=\text{son}_{v,c}$ 即可。在 Trie 的每条从根到叶子的链上，由于 dep_v 的总变化量不超过链长，故总复杂度不会超过 $O(\sum_i |s_i|)$。这个过程也说明了 $\delta(u,c)$ 的实质即为 $\text{son}_{u,c}$ 或者 u 在 fail 树的祖先中最下面的一个存在 $\text{son}_{v,c}$ 的结点。

4.2.1 字符串匹配

与 KMP 类似，给定字符串 t，AC 自动机可以对每个 i 计算 $u(i)$，满足 $s[u(i)]$ 是 $t[1,i]$ 的一个后缀，且 $u(i)$ 是这样的结点中 $s[u(i)]$ 最长的一个。这样，我们就可以在 $O(|t|)$ 的时间内在 fail 树上标记所有在 t 中出现过的字符串。然后，问题通常就会变成树上的数据结构问题。

练习 4.9. 模板题. 给定若干个不同的模式串 $P_1, P_2, \ldots, P_k$ 和一个文本串 T，对每个 P_i 计算它在 T 中的出现次数。

此外，由于字符串匹配的特殊性质，还有以下结论成立：

性质 4.10. 设 $\sum_i |P_i| = L$ 且 P_i 互不相同。

- 这些模式串在 T 中的总出现次数不超过 $2|T|\sqrt{L}$。可以据此设计一种同复杂度找出所有这些出现的算法。
- $\sum_i$ 在P_i 中出现的结点个数 $= O(L^{1.5})$

提示

练习 4.11. CF547E Mike and Friends. 给定若干个字符串 $s_1, \ldots, s_n$，令 $f(i,j)$ 为 s_j 在 s_i 中出现的次数。多次询问

$$\sum_{i=l}^{r} f(i,j)$$

练习 4.12. P5840 [COCI 2014/2015 #5. Divljak] 在上一题中将 $f(i,j)$ 改为 [s_j 在s_i 中出现过]

练习 4.13. CF587F Duff is Mad. 在上一题中将 $f(i,j)$ 改为 s_i 在 s_j 中出现的次数

解答. 设 s_i 在 Trie 上对应 u_i，$s_j[1,k]$ 在 AC 自动机上对应 $v_k^{(j)}$，那么相当于在计算

$$\sum_{i=l}^{r}\sum_{k=1}^{|s_j|}[u_i \text{ 是} v_k^{(j)} \text{ 的祖先}].$$

直观上这个东西难度和区间逆序对差不多，所以考虑一些根号的做法。

设 $L = \sum_i |s_i|$。如果 $|s_j| \le B$，那么我们暴力枚举 k 去询问。这样问题变成 qB 次询问

$$\sum_{i=l}^{r}[u_i \text{ 是} x \text{ 的祖先}].$$

离线后容易分块做到 $O(nL^\epsilon + qB)$。

对于 $|s_j| > B$ 的至多 L/B 个 j，我们可以对每个 j 在 $O(n+L)$ 时间内预处理 $f(i,j)$ 关于 i 的所有前缀和，然后 $O(1)$ 回答询问。这部分是 $O(L^2/B)$。

取 $B = L/\sqrt{q}$ 即得复杂度 $O(nL^\epsilon + L\sqrt{q})$。

练习 4.14. [JRKSJ R6] Dedicatus545. 在上一题中将求和换成 max。

练习 4.15. CF710F String Set Queries. 支持对模式串集合插入删除一个串，查询所有模式串在某个文本串 t 中出现次数的和。

练习 4.16. CF1483F Exam. 给定若干字符串 s_i，定义偏序关系 $s_i \preceq s_j$ 当且仅当 s_i 是 s_j 的子串。求有多少对 $i \neq j$ 满足 $s_i \preceq s_j$ 但是不存在其他任何 $k \neq i, j$ 使得 $s_i \preceq s_k \prec s_j$。

解答. 扫描每个 s_i 的每个前缀 $s_i[1,p]$，设它在 AC 自动机上对应 $u_{i,p}$，那么 $u_{i,p}$ 到根的所有结点都在 s_i 中出现，但是显然只有其中最长的那个 s_j 有可能贡献答案。对每个 p 都计算出这样的一个 s_j 以及对应的出现区间 $s_i[p-|s_j|+1,p]$，把这些称为候选字符串集合和候选区间集合。

当且仅当以下两种情况之一出现的时候，候选字符串集合中的一个 s_j 会不合法：

- s_j 是另一个候选字符串集合中的元素 $s_{j'}$ 的后缀
- s_j 对应的某一个候选区间是另一个候选区间的子区间

对于前者，我们在这些 s_j 对应的结点在 fail 树的虚树上简单判断即可；对于后者，我们从右往左扫一遍所有区间就能知道每个区间合不合法。可以做到线性。

练习 4.17. [HAOI2017] 字符串. 给定若干个模式串 p_i、一个文本串 s 和整数 K。对每个 p_i，计算有多少个区间 $[l,r]$ 对应的子串 $t=s[l,r]$ 满足：$|t|=|p_i|$ 且集合 $\{k : t[k] \neq p_i[k]\}$ 的极差小于 K。

解答. 条件相当于 $\mathrm{LCP}(t,p_i)+\mathrm{LCS}(t,p_i)+K \geq |p_i|$。枚举 LCP 的大小 x，则 $[l,r]$ 满足条件等价于：

- $p_i[:x]$ 以 $l+x-1$ 为结尾出现，在 AC 自动机的 fail 树上对应一个祖孙关系
- $p_i[x+K+1:]$ 以 $l+x+K$ 为开头出现，在反串的 AC 自动机的 fail 树上对应一个祖孙关系
- $p_i[:x+1]$ 不以 $l+x$ 为结尾出现

把第三条容斥掉，那么就变成若干二维数点，简单计算即可。

练习 4.18. 禁止子串. 给定若干个总长为 m 的字符串 s_i，求有多少个长为 n 的字符串 t 满足任何 s_i 都不在 t 中出现。
分别需要做到 $O(nm)$（注意不带 $|\Sigma|$）和 $O(m^2+m\log m\log n)$。

做法一. 首先考虑一个 $O(nm|\Sigma|)$ 做法，这就是在 AC 自动机上直接 DP，每次转移为 $f_u \to f_{\delta(u,c)}$，忽略所有包含某个 s_i 作为后缀的结点 $\delta(u,c)$。

可以将其改进至 $O(nm)$。考虑 $\delta(u,c)$ 的几种情况：

- $\mathrm{son}_{u,c}$ 存在，则 $\delta(u,c)=\mathrm{son}_{u,c}$
- $\mathrm{son}_{u,c}$ 不存在，则 $\delta(u,c)$ 为其 fail 树祖先中最下面的一个存在 $\mathrm{son}_{v,c}$ 的结点。

令 S_c 为 $\mathrm{son}_{v,c}$ 存在的结点集合。如果我们用填表的角度考虑 f_u，就有

$$f'_u = f_{\mathrm{fa}_u} + \sum_{c\in\Sigma}\sum_{v} f_v[\text{fail 树上}u,v\text{ 之间不存在其他}S_c\text{ 中的结点}]$$

如果令 g_u 为 f_u 在 fail 树中的子树和，$\mathrm{child}_{u,c}$ 为 S_c 在 fail 树上的虚树中 u 的所有儿子，那么

$$f'_u = f_{\mathrm{fa}_u} + \sum_{c\in\Sigma:\mathrm{son}_{u,c}\text{ 存在}} \left(g_u - \sum_{v\in\mathrm{child}_{u,c}} g_v \right).$$

（对于根，c 的枚举范围则为整个 Σ）

据此转移即可得到一个 $O(nm)$ 的做法。

做法二.　首先由矩阵快速幂可以得到一个 $O(m^3 \log n)$ 的做法。设转移矩阵为 $\mathbf{A}$，初始 DP 状态为 $\mathbf{f}_0$，则答案即为 $a_n = \mathbf{1}^T\mathbf{A}^n\mathbf{f}_0$。

由 Cayley-Hamilton 定理，$\mathbf{A}$ 的特征多项式 $p(\lambda) = \sum_{i=0}^{m} c_i\lambda^i$ 满足 $p(\mathbf{A}) = \mathbf{0}$。乘以 $\mathbf{A}^{n-m}$，再乘上 $\mathbf{1}^T$ 和 $\mathbf{f}_0$ 就得到

$$\sum_{i=0}^{m} c_i a_{n-m+i} = 0.$$

也就是说 a 满足一个 m 阶线性递推。

我们可以用做法一 $O(m^2)$ 计算出前 $2m$ 项，然后用 BM 算法求出线性递推系数，然后使用线性递推即可得到一个 $O(m^2 + m\log m\log n)$ 的做法。

做法三.　我们知道单串情况下有一个使用生成函数的做法可以做到 $O(m\log m\log n)$。对于多串情况，我们可以做一些类似的事情。

不妨假设串之间都没有包含关系。令 $F(x)$ 为所有合法串的生成函数，$G(x)$ 为在末尾首次出现了某个 $s\in L$ 的字符串的生成函数，$G_i(x)$ 为在末尾首次出现了某个 $s\in L$ 且 $s = s_i$ 的字符串生成函数。记 $\sigma = |\Sigma|$，则有

$$\begin{aligned}
1 + F(x)\cdot\sigma x &= F(x) + G(x)\\
G(x) &= \sum_i G_i(x)\\
F(x)x^{|s_i|} &= \sum_j G_j(x)\cdot C_{i,j}(x)\\
C_{i,j}(x) &= \sum_{k=1}^{\min(|s_i|,|s_j|)} [s_j[|s_j|-k+1,|s_j|] = s_i[1,k]]x^{|s_i|-k}.
\end{aligned}$$

解一下这个方程可以得到

$$F(x) = \frac{1}{1-\sigma x + \mathbf{l}(x)^T\mathbf{C}^{-1}(x)\mathbf{1}},$$

其中 $\mathbf{l}(x) = (x^{|s_i|})_i$。

在字符串个数 k 比较小的时候，计算 $C^{-1}(x)$ 可以做到 $\tilde{O}(k^3m)$，这会比上一种做法更好。

5 后缀结构

5.1 后缀树

对于字符串 s，由 s 的所有后缀构建出的压缩 Trie 称为 s 的隐式后缀树。如果额外保留每个后缀的结尾结点，则得到 s 的后缀树。由于任何子串都是一个后缀的前缀，后缀树可以处理相当多的子串问题。

5.1.1 构建

在本节中，我们讨论支持向后加字符并维护隐式后缀树的 Ukkonen 算法。

首先，如果直接使用压缩 Trie 的构建算法，我们可以得到下面这样一个暴力：

算法 5.1. 暴力构建后缀树. 压缩 Trie 的每条边上维护这条边对应的字符串在 s 中的位置。考虑在右边加一个字符后发生的变化。这对应于所有的后缀都延伸一步，在压缩 Trie 上有三种情况：

- 如果一个后缀处于叶子结点上，那么只需要隐式地把父边伸长
- 如果一个后缀处于非叶结点上，那么可能需要新建子结点表示延伸后的该后缀
- 如果一个后缀处于一条边上，那么还可能需要分裂这条边，然后再进行上一条

我们把后两种情况统称为“不处于叶子结点上”。根据后缀的性质，如果后缀 $s[i:]$ 不处于叶子结点上，那么它是某个 $s[i':]$ 的真前缀。那么对任何 $j \geq i$，后缀 $s[j:]$ 将是 $j' = i' + (j - i)$ 对应的后缀 $s[j':]$ 的真前缀，从而 $s[j:]$ 也处于非叶结点上。也就是说任意时刻可以把所有后缀 $s[i:]$ 按照 i 分成两部分，前半部分都处于叶子结点上，后半部分都不处于叶子结点上。我们维护后半部分的开始位置 k，每次判断一下 $s[k:]$ 在末尾添加字符后需不需要发生变化，如果需要那么就做相应的修改，然后令 $k \leftarrow k + 1$ 并重复此过程；如果不需要做那么就终止。

上述过程的瓶颈在于找到 $s[k:]$ 在压缩 Trie 上的位置，或者更具体地说，根据 $s[k:]$ 的位置找到 $s[k+1:]$ 的位置。这可以通过后缀链接来实现：对压缩 Trie 上的每个非叶结点 u，我们维护 `fail[u]` 使得 `fail[u]` 指向的结点对应的字符串恰好为 u 对应的字符串去掉首字母。由于 u 是分叉点，根据后缀的性质，u 去掉首字母后仍然是分叉点，故这样的结点存在。假设 $s[k:]$ 对应的位置为 u 沿着 $s[|s| - l + 1]$ 对应的出边走 l 长度，那么 $s[k+1:]$ 对应的位置可以从 `fail[u]` 出发走 l 长度得到（注意可能经过多条压缩 Trie 上的边）。

只有分裂边的情况下需要计算对 k 新建的结点的后缀链接，这即是 `fail[u]` 出发走 l 长度到达的位置。不过具体实现的时候还有很多 trick 让代码很短。

代码 5.1: Ukkonen 算法构建后缀树

```
1 // source: oi wiki
2 struct SuffixTree {
3   // (start,len): 结点父边对应字符串的 [start,start+len) 区间
4   // link: 后缀链接
```

```
5   int ch[M + 5][RNG + 1], start[M + 5], len[M + 5], link[M + 5];
6   int s[N + 5];
7
8   // root = 1
9   // (cur, rem) 表示当前 s[k:] 的位置为 cur 沿着 s[|s|-rem+1] 对应的边走
        rem 长度到达的结点
10  int cur{1}, rem{0}, n{0}, tot{1};
11
12  SuffixTree() {
13    len[0] = inf; // 0 是空节点，防止不存在的边被 while 走进去
14  }
15
16  int new_node(int start, int le) {
17    ++tot;
18    start[tot] = start;
19    len[tot] = le;
20    return tot;
21  }
22
23  void extend(int x) {
24    s[++n] = x;
25    ++rem;
26
27    // lst 是上一轮添加的叶子的父结点。
28    // 实际上只有 lst 由分裂产生时才需要更新 link，这里为了代码简单写成这
          样
29    for (int lst{1}; rem;) {
30        // 从 cur 出发走 rem 长度到达的位置
31      while (rem > len[ch[cur][s[n - rem + 1]]])
32        rem -= len[cur = ch[cur][s[n - rem + 1]]];
33
34      int &v{ch[cur][s[n - rem + 1]]};
35      // 所在位置为 (cur,v] 边上
36
37      int c{s[start[v] + rem - 1]};
38
39      if (!v || x == c) {
40        lst = link[lst] = cur;
41        if (!v)
42          v = new_node(n, inf); // 落在点上，加叶子
43        else
44          break;                // 已经存在，结束
45      } else {
46        // 分裂当前边并加叶子
47        int u{new_node(start[v], rem - 1)};
```

```
48          ch[u][c] = v;
49          ch[u][x] = new_node(n, inf);
50          start[v] += rem - 1;
51          len[v] -= rem - 1;
52
53          // link[lst] = u, 同时把 cur 的儿子改成 u
54          lst = link[lst] = v = u;
55        }
56
57        // 下一个后缀
58        if (cur == 1)
59          --rem;
60        else
61          cur = link[cur];
62      }
63    }
64  } Tree;
```

评注. 如果想要后缀树而不是隐式后缀树的话还得在末尾加一个特殊字符，或者基于相同的道理再写一遍差不多的代码。

这个算法又慢又难写，如无必要不如直接写 *SAM*。

5.1.2　应用

由于后缀树是压缩 Trie，我们在 Trie 上思考问题，然后将其应用到压缩 Trie 上即可。

练习 5.2.

- 计算字符串 s 的本质不同子串个数、总长度等信息
- 多次询问：给定字符串 t，求 t 在 s 中的出现次数/第一次出现位置/$[l,r]$ 中的第一次出现位置和出现次数
- [TJOI2015] 弦论 求字典序第 k 小的本质不同子串和子串。也可以多次询问一些可简单表示的信息（如长度）
- 求所有循环表示中字典序最小的一个
- 快速计算两个后缀的 LCP，快速计算两个子串的 LCP

5.1.3　广义后缀树

给定多个串，对所有串的所有后缀建立的压缩 Trie 称为广义后缀树。容易发现 Ukkonen 算法的原理天然支持多串，只不过要在每个串构建完成后额外做一些处理来将所有叶子“封闭”。广义后缀树可以处理多个串的一些子串问题。

例 5.3. 给定若干字符串，求它们的最长公共子串。

5.2　后缀数组

后缀数组是对 s 的所有后缀按字典序排序后得到的数组，也就是 $\texttt{sa}[i]$ 表示字典序第 i 小的后缀。其逆排列 $\texttt{rk}[i]$ 表示后缀 i 的排名。

根据定义，这就是所有后缀在后缀树上的 DFS 序，因此可以根据后缀树来构建。也可以直接使用倍增配合双关键字计数排序来得到。还有一些 $O(n)$ 的常数更好的算法，不过 OI 中不必要。

记 $\texttt{height}[i] = \mathrm{LCP}(\texttt{sa}[i], \texttt{sa}[i-1])$，则 $\mathrm{LCP}(\texttt{sa}[i], \texttt{sa}[j]) = \min_{i<k\le j} \texttt{height}[k]$。可以据此快速计算 LCP。由于 $\texttt{height}[rk[i]] \ge \texttt{height}[rk[i-1]] - 1$，可以简单 $O(n)$ 计算。

通常认为后缀数组在功能上不强于后缀树。

5.3　后缀自动机

s 的后缀自动机（Suffix Automaton, SAM）是接受 s 的所有后缀的**最小** DFA。另一方面，由于后缀 Trie 也是一个接受 s 所有后缀的 DFA，故后缀自动机是后缀 Trie 的最小化。

由 Myhill-Nerode 定理，最小 DFA 可以由向右加字符形成的等价类刻画。对于所有子串，这个等价类可以由以下断言刻画：

断言 5.4. 定义 $\mathrm{endpos}(t)$ 为字符串 t 在 s 中所有出现的结尾位置构成的集合。两个字符串等价当且仅当其 endpos 相同。令等价类的代表元为其中最长的串，等价类内的所有元素即为代表元的长度不低于某个值的所有后缀。

对照后缀树的性质，我们还可以得到

断言 5.5. 两个字符串等价当且仅当其在**反串**的后缀树上处于同一条边上。因此，后缀自动机的结点个数与反串后缀树的结点个数相同，不超过 $2n-1$。

评注. *很快我们会看到可以在构建后缀自动机的同时构建反串后缀树。*

5.3.1　构建

本节讨论支持向后加字符的 Blumer 算法，它同时构造出反串的后缀树（也称为 parent 树）。于是我们也得到了一个支持向前加字符的后缀树构建算法。

评注. *存在支持双向加字符并维护隐式后缀树的算法。*

记 $\delta(u, c)$ 为状态 u 在末尾添加 c 后转移到的状态，后缀链接 $\texttt{link}[u]$ 指向 u 在反串后缀树上的父亲（也即 u 对应的字符串不断去掉首字母到达的首个非 u 状态）。考虑新增字符的影响。由于 $s + c$ 的所有后缀即为 s 的所有后缀在末尾添加 c 得到的，故受影响的状态应该为 s 所在状态在 link 树上到根的链上每个状态 u 转移 c 得到的状态，这些状态也处在一条 link 链上。根据受影响的情况，可以分为以下三类：

- 在链的下端，$\delta(u, c)$ 不存在。此时，这些 u 添加 c 后都对应着同一个 endpos。于是应该新建一个结点，然后让这些 u 的 $\delta(u, c)$ 都指向新结点。

- 在链的上端，$\delta(u,c)$ 已经存在。链会分为若干段，除了最下方的一段外，每一段内 $\delta(u,c)$ 都指向同一个 v，且 v 等价类即是这些 u 等价类的并在末尾加入 c 得到的。对于这些段来说，什么都不需要做
- 对于最下方的一段，v 所在等价类的最长串可能比 u 添加 c 得到的串还要长，而这些过长的串的 endpos 不会更新。在此情况下，应该将这些过长的串**分裂**出去。

据此直接实现即可。

代码 5.2: Blumer 算法构建 SAM

```
void ins(int c)
{
    // a[u].ch[c] 为 delta(u,c), a[u].fa 为 link[u]
    // cur 为先前的 s 所在的状态

    // 为链的下端创建新结点
    int u=cur;
    cur=++node_cnt;
    a[cur].len=a[u].len+1;
    // 将 delta(u,c) 不存在的 u 的 delta(u,c) 都指向 cur
    for(;u&&!a[u].ch[c];u=a[u].fa)a[u].ch[c]=cur;
    if(!u){a[cur].fa=1;return;}
    int v=a[u].ch[c];
    // 判断是否有过长的串
    if(a[v].len==a[u].len+1){a[cur].fa=v;return;}
    // 分裂结点，v 变成过长的串对应的等价类，用新结点 t 替换原来的 v
    int t=++node_cnt;
    a[t]=a[v];
    a[t].len=a[u].len+1;
    a[v].fa=t,a[cur].fa=t;
    // 让最下方的一段指向新结点 t
    for(;u&&a[u].ch[c]==v;u=a[u].fa)a[u].ch[c]=t;
}
```

定理 5.6. SAM 的转移边数量不超过 $3n-2$

证明. 把结点分为出度为 1 的和出度大于 1 的。如果一个状态的出度大于 1，那么其代表元在**正串**后缀 Trie 上对应一个分叉点，且分叉数量至少为出度。所以总出度不超过点数加上后缀树的叶结点数量减一，也就不超过 $3n-2$。 □

评注. 紧上界是 $3n-4$，但是这个看起来好理解很多。

5.3.2　广义后缀自动机

可以用同样的定义将 SAM 扩展到多串甚至 Trie 上（Trie 的每个结点代表从根到它的路径形成的字符串）。Blumer 构造算法也可以扩展到多串和 Trie 的情况，仅有的区别在于

$\delta(cur, c)$ 可能已经存在，此时无需创建新结点。

代码 5.3: Blumer 算法构建广义 SAM

```
// 在某个串/Trie 结点后添加字符 c
// cur 为添加前该串/Trie 结点对应的状态
// 例如，多串情况下，在开始处理一个串之前应将 cur 设为根结点
void ins(int c)
{
        int u=cur;
        if(a[u].ch[c])
        {
                int v=a[u].ch[c];
                if(a[v].len==a[u].len+1){cur=v;return;}
                int t=++node_cnt;a[t]=a[v];a[t].len=a[u].len+1;
                cur=a[v].fa=t;
                for(;u&&a[u].ch[c]==v;u=a[u].fa)a[u].ch[c]=t;
                return;
        }
        cur=++node_cnt;a[cur].len=a[u].len+1;
        for(;u&&!a[u].ch[c];u=a[u].fa)a[u].ch[c]=cur;
        if(!u){a[cur].fa=1;return;}
        int v=a[u].ch[c];
        if(a[v].len==a[u].len+1){a[cur].fa=v;return;}
        int t=++node_cnt;a[t]=a[v];a[t].len=a[u].len+1;
        a[cur].fa=a[v].fa=t;
        for(;u&&a[u].ch[c]==v;u=a[u].fa)a[u].ch[c]=t;
}
```

先前的复杂度分析同样适用。广义 SAM 的后缀链接树是反串的广义后缀树。

与 AC 自动机类似，在广义 SAM 上也有以下结论成立：

性质 5.7. 设 $\sum_i |s_i| = L$，则 $\sum_i$ 在s_i 中出现的结点个数 $= O(L^{1.5})$。

5.3.3　应用

SAM 是最常用的构建后缀树的方式。绝大多数时候我们都没有使用自动机结构，而是只使用后缀树。不过，自动机结构也提供了解决问题的另一个视角：DAG 的路径与字符串的子串一一对应。

拜此所赐，绝大多数 OIer 想象后缀树的时候都是反的……？

练习 5.8. 给定串 s，维护串 t，支持在串 t 的开头插入/删除字符，结尾加入字符，维护 t 所在的 SAM 状态（保证 t 永远是 s 的子串）。$O(1)$。

练习 5.9. [NOI2018] 你的名字. 固定串 s。每次询问给定字符串 t 和 s 的子串 $s[l, r]$，求 t 的本质不同子串中有多少个不是 $s[l, r]$ 的子串。

解答. 容斥成是 $s[l,r]$ 的子串。枚举 t 的前缀 i，双指针计算最小的 j_i 使得 $t[j_i,i]$ 是 $s[l,r]$ 的子串，这可以通过在 s 的 SAM 上走并配合 parent 树来判断是否合法。在 parent 树上找到 $t[j_i,i]$ 对应的结点，最后算一下链并即可。也可以在 t 的 SAM 上一起走，这样可以在 t 的 SAM 上暴力算这个链并。

5.4 练习

练习 5.10. CF1276F. 给定字符串 s，令 $s^{(i)}$ 为把 $s[i]$ 替换为特殊字符 $*$ 得到的字符串，求 $s, s^{(1)}, s^{(2)}, \ldots, s^{(n)}$ 的本质不同子串个数，也就是求有多少个字符串至少作为其中一个的子串出现。

练习 5.11. P5576 [CmdOI2019] 口头禅. 给定 n 个字符串 $s_1, \ldots, s_n$，多次询问编号在 $[l,r]$ 内的字符串的最长公共子串。

解答. 构建广义后缀树，然后需要在树上找到一个对应字符串最长的结点，满足其子树内包含了 $[l,r]$ 所有颜色。维护一个可能贡献答案的集合 S，在树上合并计算每个结点包含的所有颜色连续段的时候，如果连续段发生了合并，那么就在 S 中加入新的连续段，对应权值为此结点的字符串长度。

查询时需要在 S 中寻找包含 $[l,r]$ 的区间的最大权值，二维数点即可。

练习 5.12. [NOI2016] 优秀的拆分. 计算字符串 s 有多少个形如 AABB 的子串。

聪明的做法. 只需要对每个 i 计算出以 i 为开头/结尾的 AA 数量。

枚举 A 的长度 l，将序列按 l 分块。假设 i 所在块的左端点为 p，有一个以 i 为结尾的 AA 当且仅当 $\mathrm{LCP}(s[p-l:], s[p:]) \geq i-p+1$ 且 $\mathrm{LCS}(s[:p-l], s[:p]) \geq p+l-i$。那么只要我们算出 $x = \mathrm{LCP}(s[p-l:], s[p:])$ 和 $y = \mathrm{LCS}(s[:p-l], s[:p])$ 就能知道有贡献的 i 的范围了。总共需要 $O(n \log n)$ 次求 LCP 和 LCS。

不太聪明的做法. 还是对每个 i 计算以 i 为开头/结尾的 AA 的数量。以结尾为例，只要对每个 i 计算有多少个 $j < i$ 满足 $\mathrm{LCP}(s[j:], s[i:]) \geq i-j$ 即可。这可以在后缀树上 dsu on tree 完成。$O(n \log^2 n)$。

评注. 还有一个做法是用 *runs*。

练习 5.13. P5115 Check,Check,Check one two!. 给定函数 f 和 g，计算

$$\sum_{1 \leq i < j \leq n} f(\mathrm{LCP}(s[i:], s[j:])) \cdot g(\mathrm{LCS}(s[:i], s[:j]))$$

解答. 不妨设 $f(0) = g(0) = 0$。差分，设 $\hat{f}(i) = f(i) - f(i-1)$，$\hat{g}(i) = g(i) - g(i-1)$，则原

式变为

$$\begin{aligned}
&\sum_{1\le i<j\le n}\left(\sum_{k\ge 1}\hat{f}(k)[\mathrm{LCP}(s[i:],s[j:])\ge k]\right)\left(\sum_{l\ge 1}\hat{g}(l)[\mathrm{LCS}(s[:i],s[:j])\ge l]\right)\\
&=\sum_{k,l\ge 1}\hat{f}(k)\hat{g}(l)\sum_{1\le i<j\le n}[\mathrm{LCP}(s[i:],s[j:])\ge k][\mathrm{LCS}(s[:i],s[:j])\ge l]\\
&=\sum_{k,l\ge 1}\hat{f}(k)\hat{g}(l)\sum_{l\le i<j\le n-k+1}[s[i-l+1:i+k-1]=s[j-l+1:j+k-1]]\\
&=\sum_{1\le i<j\le n}\sum_{d\ge 0}[s[i:i+d]=s[j:j+d]]\sum_{k+l-2=d}\hat{f}(k)\hat{g}(l)\\
&=\sum_{t}\binom{t\text{ 在}s\text{ 中的出现次数}}{2}\cdot h(|t|-1)
\end{aligned}$$

预处理出 h 及其前缀和后在后缀树上简单计算即可。

5.5　基本子串结构

暂时不会。但是应该是非常强大的工具，有待 OI 进一步开发。

5.5.1　压缩后缀自动机

6 回文结构

如果 $s = s^R$，则称 s 为回文串。容易用各种方法在线性时间内判断一个串是否是回文串。由定义，回文串满足 $s[l,r] = s[-r,-l]^R$。反过来，s 是回文串当且仅当存在若干并为 $[1,|s|]$ 的区间 $[l_i,r_i]$ 满足 $s[l_i,r_i] = s[-r_i,-l_i]^R$ 对所有 i 成立，这给出了回文串的一个判据。

然而单带图灵机上需要 $\Omega(n^2)$ 时间来判定回文串。

6.1 Manacher 算法

Manacher 算法可以在 $O(|s|)$ 的时间内计算出字符串 s 以每个位置（包括两个字符之间）为中心能扩展出的最长串。方便起见，可以在字符串的每两个字符之间插入特殊字符，这样就不需要考虑相邻字符之间的位置作为中心的情况。设 h_i 为以 i 为中心的最长回文半径，即最大的满足 $s[i-h+1,i+h-1]$ 为回文串的 h。

算法 6.1. Manacher. 从左到右扫描 i 并维护最大的 $i+h_i$。假设目前的最值在 j 处取到。当 $i-1$ 移动到 i 时，由于 $s[j-h_j+1,j+h_j-1]$ 是回文串，i 在 $r<j+h_j-i$ 半径内的子串 $s[i-r+1,i+r-1]$ 等于其对称点 $i'=2j-i$ 对应的子串的反串 $s[i'-r+1,i'+r-1]$。因此，$h_i \geq \min(j+h_j-i,h_{i'})$。从这个下界开始暴力尝试扩展即可。
注意到只有 $h_{i'} \geq j+h_j-i$ 时才有可能发生扩展，此时 h_i 的初值为 $j+h_j-i$，于是每次扩展都会增大 $i+h_i$ 的最大值。而 $i+h_i \leq |s|+1$，于是整个过程中总共至多扩展 $|s|$ 次。

练习 6.2. 支持在线末尾添加字符，查询某个 h_i。

练习 6.3. 求出以每个点为结尾的最长回文子串。

解答. 在 Manacher 算法首次将 $i+h_i-1$ 扩展到 k 时，以 k 为结尾的最长回文串即为 $s[i-h_i+1,i+h_i-1]$。

练习 6.4. 求出以每个点为结尾的回文子串数量。

练习 6.5. [NOI2023] 字符串. 给定字符串 s，每次询问给定 i,r（保证 $i+2r-1 \leq |s|$），求有多少个 $1 \leq l \leq r$ 满足 $s[i:i+l-1] < s[i+l:i+2l-1]^R$。

解答. 令 $s' = ss^R$，则 $s[i:i+l-1] < s[i+l:i+2l-1]^R$ 当且仅当

- $s'[i:] < s'[i'_l:]$，其中 $i'_l = |s|+|s|-(i+2l-1)+1$
- $s[i:i+l-1] \neq s[i+l:i+2l-1]^R$

容斥掉第二个限制，那么分别需要计算 $s'[i:] < s'[i'_l:]$ 的 l 的数量以及 $s'[i:] < s'[i'_l:]$ 且 $s[i:i+l-1] = s[i+l:i+2l-1]^R$ 的 l 的数量。前者二维数点即可，下面考虑后者。

设 $h = h_{i+l-1}$ 为最大的满足 $s[i+l-h:i+l-1] = s[i+l:i+l+h-1]$ 的 h（即回文半径），则在 $s[i:i+l-1] = s[i+l:i+2l-1]^R$ 也即 $h \geq l$ 的情况下，$s'[i:] < s'[i'_l:]$ 等价于 $s[i+l+h] < s[i+l-1-h]$。那么我们要算的就是满足 $h_{i+l-1} \geq l$ 且

$s[i+l+h_{i+l-1}] < s[i+l-1-h_{i+l-1}]$ 的 l 的数量。第二个条件只与 $i+l-1$ 有关，可以记作 P_{i+l-1}。设 $x=i+l-1$，那么就是要算 $h_x \geq x-i+1$、$i \leq x \leq i+r-1$ 的 P_x 的和，二维数点即可。

6.2　回文树

s 的回文树（Palindromic Tree，亦称回文自动机（Palindromic Automaton，PAM）、EERTREE）是一个接受 s 所有回文子串的**后一半**（上取整）的树形 DFA。s 的本质不同回文子串和回文树的非根结点一一对应，因此后面我们将混用结点与其对应的字符串。$\delta(u,c)$ 指向 u **双侧**添加 c 后得到的回文串。不难发现这样转移时得到的回文串的奇偶性都是相同的，因此我们使用两个根——偶根和奇根，分别能够转移到对应偶数和奇数长度的回文串。由于只有 S 能转移到 cSc，每个非根结点都恰好有一条入边，故 δ 构成了两棵树。

我看到很多资料写成 EER Tree，这看起来不知所云

回文树同样配备后缀链接 `link`，`link`$[u]$ 指向 u 的最长回文真后缀，偶根的 `link` 指向奇根。我们不难根据 Blumer SAM 构建算法的经验写出支持向后添加字符并维护回文树的算法。

算法 6.6. 回文树构建算法. 记 $\mathrm{len}(u)$ 表示 u 对应的回文串的长度。特别地，偶根和奇根的长度分别为 0 和 -1。

设当前字符串 s 最长回文后缀对应的状态为 u，考虑在末尾加入 c 之后发生的变化。由于 $s+c$ 的回文后缀一定是 s 的回文后缀在头尾添加 c 得到的（注意反过来未必），故我们可以让 u 沿着 `link` 链向上跳，直到找到一个满足 $s[|s|-\mathrm{len}(u)]=c$ 的 u。此时，$\delta(u,c)$ 应为 $s+c$ 的最长回文后缀。如果 $\delta(u,c)$ 已经存在，那么无需处理；否则，我们需要新建结点 v 并让 $\delta(u,c)=v$，还需要计算 `link`$[v]$，这可以用同样的方法完成。

代码 6.1: 回文树构建算法

```
void clear()
{
        node_cnt=cur=1;
        a[1].len=-1,a[0].len=0;a[0].fa=1;
}
void ins(int c,int pos)
{
        int u=cur;
        for(;st[pos]!=st[pos-a[u].len-1];u=a[u].fa);
        if(a[u].ch[c]){cur=a[u].ch[c];return;}
        cur=++node_cnt;
        a[node_cnt].len=a[u].len+2;
        int v=a[u].fa;
        for(;st[pos]!=st[pos-a[v].len-1];v=a[v].fa);
        a[cur].fa=a[v].ch[c];
        a[u].ch[c]=cur;
}
```

定理 6.7. 回文树的构建算法的时间复杂度为 $O(|s|)$。这也说明了本质不同回文子串的数量不超过 $|s|$。

证明. 算法可以视为维护了两个指针 u, v，其中 u 为当前 s 的最长回文后缀，v 为 u 的最长回文真后缀。每次添加字符时，u, v 的长度都至多增加 2，每次上跳都会减少长度，于是总复杂度不超过 $O(|s|)$。 □

练习 6.8. 设计一个支持双向加字符的回文树构建算法。

练习 6.9. 支持向后加字符，查询

- 以某个点为结尾的最长回文子串/回文子串数量
- 本质不同回文子串数量
- $s[l, r]$ 的最长回文后缀
- 判断 $s[l, r]$ 是否是回文子串，如果是则查询出现次数

练习 6.10. 设计一个 $O(n|\Sigma|)$ 对 Trie 构建回文树的算法。

6.3　回文与 Border

事实 6.11. 对于回文串 s，$s[: i]$ 是一个回文前缀当且仅当它是 s 的一个 border。后缀同理。

因此，回文前/后缀与 border 具有相同的性质。

引理 6.12. 如果 $i \geq |s|/2$ 且 $s[1 : i]$ 是一个 border，那么 s 是回文串当且仅当 $s[1 : i]$ 是回文串。

引理 6.13. 回文串的回文后缀长度构成至多 $\lceil \log_2 |s| \rceil$ 段等差数列。

因此，回文树的 `link` 链具有与 KMP 中的 `fail` 链类似的性质。

练习 6.14. [SHOI2011] 双倍回文. 给定字符串 s，对每个 i 求出最长的满足 u 和 uu 都是 $s[1 : i]$ 回文后缀的 u。

练习 6.15. 支持在末尾添加一个字符，查询所有回文后缀中心位置的权值和。

解答. 所有回文后缀即为 `link` 链上的结点。维护所有回文后缀的中心位置 S。考虑添加字符 c 后回文后缀发生的变化，之前的链上有一些可以转移 c，有一些则不能，而前者即对应着所有新的回文后缀，这些新回文后缀的中心位置不会发生变化。我们可以找到所有不能转移 c

的结点，将他们从 S 中删除。由于每次扩展至多增加一个新的回文后缀，这个过程的总复杂度是 $O(|s|)$ 的。

如何找到所有不能转移 c 的结点？注意到每条 `link` 边实际上都代表了一个字符：$\texttt{link}[u]$ 作为 u 的后缀时左边的字符。`link` 链上的一个结点能转移 c 当且仅当它下面那条 `link` 边代表的字符是 c。于是我们可以维护每条边上方第一条与它代表的字符不同的边，这样就可以 O(个数) 找到所有需要删除的结点。

练习 6.16. 设计一个添加字符复杂度为均摊 $O(1)$ 且严格 $O(\log n)$ 的回文树构建算法。

解答. `link` 链对应着最下方结点的 border 链，$p_u = \text{len}(u) - \text{len}(\texttt{link}[u])$ 对应着 u 的最小周期。可以根据 p_u 将 `link` 链划分成若干段，一段内 `link` 边代表的字符都是相同的。根据推论 2.14，所有长度 $\geq |u|/2$ 的 border 都在同一段内，且再向上走一条边后长度必然 $< |u|/2$。因此，可以维护每个点所在段的顶部，每次失配后直接跳到段顶。

例 6.17. 支持在末尾添加一个字符，查询所有回文后缀开头位置的权值和。

解答. 对回文树的每个结点维护其最后一次作为一个 p_u 段的最下方结点时，这个段（不含最上方结点）对应的回文后缀开头位置的权值和 f_u。每次添加字符后暴力更新所有 $O(\log n)$ 段的最下方结点。

设一个段的最下方结点为 u，段的长度定义为包含的边数。如果段的长度为 1，那么 f_u 只包含 u 本身的贡献；如果长度 ≥ 2，我们说明 f_u 可以由 $v = \texttt{link}[u]$ 的 f 添加一个位置的贡献得到。

断言 6.18. 假设目前整个串为 s，则 v 最后一次出现的结尾为 $|s| - p_u$，且在那次出现时作为段底被更新过 f。

证明. 由于段长度至少为 2，$|v| \geq |u|/2$。由于 v 是 u 的 border，故 v 出现在 $u[:|v|]$ 和 $u[p_u + 1:]$。根据引理 2.29，v 在 $u[d+1:d+|v|]$ 出现当且仅当 $\gcd(d, p_u)$ 是 v 的周期，而 p_u 已经是 v 的最小周期，故 v 在 u 中没有其它出现，即 v 的最后一次出现的结尾为 $|s| - p_u$。

v 在 $|s| - p_u$ 出现时作为段底当且仅当那时不存在一个长为 $|v| + p_u$ 的回文后缀，而这样的回文后缀只能是 u。如果 u 在那时是一个回文后缀，那么 $s[-p_u - |u| :]$ 将是一个回文串，从而 u 不是目前的一个段底，矛盾。因此 v 在 $|s| - p_u$ 出现时作为段底被更新过 f。 □

$|s| - p_u$ 时刻 v 到段顶对应的那些开头位置为 $\{|s| - p_u - |v| + kp_u : 0 \leq k < L_v\}$，其中 L_v 为 v 到段顶的边数，而 $|s|$ 时刻 u 到段顶对应的那些开头位置为 $\{|s| - |u| + kp_u : 0 \leq k < L_u\}$。注意 $|u| = |v| + p_u$，$L_u = L_v + 1$，故这两个集合的差别只有一个元素——段顶下面的结点对应的开头位置。

练习 6.19. 给定字符串 s，计算其最少能划分成多少段回文串，并计算方案数。

练习 6.20. CF932G Palindrome Partition. 将 s 划分为尽量少的子串 $s = s_1 \cdots s_k$，使得 k 是偶数且 $s_i = s_{k-i+1}$。

解答. 首先需要 $|s|$ 是偶数。令 $s' = s[1]s[-1]s[2]s[-2]\cdots s[|s|/2]s[-|s|/2]$，则 s 的合法划分等价于 s' 的偶回文串划分。

7 Lyndon Words 和 Runs

暂时不会。

参考文献

本讲义主要由我个人写作，整理了许多我最近重新学习的字符串知识。以下是一些过去，现在和未来参考的资料。

ChatGPT 5.5 (Thinking xhigh) 给出了许多结论的简洁证明，与我讨论了许多问题，并对内容进行了审阅。强烈推荐大家使用 Gemini Pro (AI Studio)、ChatGPT Plus 或以上水平的 AI 工具辅助学习。

OI Wiki.

ix35. 字符串技术巡礼. 2023.

清烛的题单.

7.1 Border，周期与字符串匹配

- command_block. Border 理论小记. 2020.。
- RandomShuffle. Border 理论. 2024. 。
- qwaszx. border 的亿点研究. 2020.。

7.2 自动机

徐哲安.《浅谈有限状态自动机及其应用》. 信息学奥林匹克中国国家集训队论文, 2021。

7.3 Trie 和 AC 自动机

- abruce 的题单.
- qwaszx. 一个 AC 自动机的 trick. 2021.

7.4 后缀结构

- command_block 的题单.
- 徐翊轩.《浅谈压缩后缀自动机》. 信息学奥林匹克中国国家集训队论文, 2020.
- 许庭强.《一类基础子串数据结构》. 信息学奥林匹克中国国家集训队论文, 2023.
- Crashed.《一类基础子串数据结构》摘抄及注解. 2023.
- forest114514. 字符串学习笔记. 2024.

7.5 回文结构

- hyfhaha 的题单